A

PROJECT REPORT

ON

Wellness Prediction Suite

Submitted in partial fulfilment of the requirements

For the award of a Degree of

BACHELOR OF ENGINEERING

IN

CSE(AIML)

Submitted By

Aneesh Kashetty	**245320748020**
MVR Ruthwik	**245320748027**
Praneeth Kumar Katta	**245320748306**

Under the guidance of

Ms. R Koteshwaramma

Associate Professor

Department of CSE(AIML)

NEIL GOGTE INSTITUTE OF TECHNOLOGY

Kachavanisingaram Village, Hyderabad, Telangana 500058.

JUNE 2024

NEIL GOGTE INSTITUTE OF TECHNOLOGY

A Unit of Keshav Memorial Technical Education (KMTES)

Approved by AICTE, New Delhi & Affiliated to Osmania University, Hyderabad

CERTIFICATE

This is to certify that the Major project work entitled "**Wellness Prediction Suite"** *is a bonafide work carried out by* **Aneesh Kashetty (245320748020), MVR Ruthwik (245320748027), Praneeth Kumar Katta (245320748306)** of **IV** year **VIII** semester **Bachelor of Engineering** *in CSE(AIML) by Osmania University, Hyderabad during the academic year* **2023-2024** *is a record of bonafide work carried out by them. The results embodied in this report have not been submitted to any other University or Institution for the award of any degree.*

Internal Guide
Ms. R Koteswaramma

Head of Department
Dr. T. Prem Chander

External

NEIL GOGTE INSTITUTE OF TECHNOLOGY
A Unit of Keshav Memorial Technical Education (KMTES)
Approved by AICTE, New Delhi & Affiliated to Osmania University

DECLARATION

We hereby declare that the Major Project Report entitled, "**Wellness Prediction Suite**" submitted for the B.E degree is entirely our work and all ideas and references have been duly acknowledged.

It does not contain any work for the award of any other degree.

Date:

Aneesh Kashetty	**245320748020**
MVR Ruthwik	**245320748027**
Praneeth Kumar Katta	**245320748306**

ACKNOWLEDGEMENT

We are happy to express our deep sense of gratitude to the **Principal of the college, Prof. R. Shyam Sunder**, Neil Gogte Institute of Technology, for having provided us with adequate facilities to pursue our project.

We would like to thank **Dr. T. Prem Chander**, **Head of the Department,** CSE(AIML) , Neil Gogte Institute of Technology, for having provided the freedom to use all the facilities available in the department, especially the laboratories and the library.

We would also like to thank my internal guide **Ms. R Koteswaramma**, **Head of the Department** for his technical guidance & constant encouragement.

We sincerely thank our seniors and all the teaching and non-teaching staff of the Department of CSE(AIML) for their timely suggestions, healthy criticism, and motivation during this work.

Finally, we express our immense gratitude with pleasure to the other individuals who have either directly or indirectly contributed to our need at the right time for the development and success of this work.

ABSTRACT

Anomaly detection, a fundamental problem in various domains including machine learning, computer vision, anomaly detection, and signal processing, refers to the identification of previously unseen patterns or instances within a given dataset. This abstract explores the state-of-the-art techniques and methodologies employed in anomaly detection, highlighting the challenges, advancements, and applications across diverse fields. We provide a comprehensive overview of traditional approaches such as statistical methods, nearest neighbour-based techniques, and density estimation, as well as recent advancements in deep learning-based methodologies including autoencoders, generative adversarial networks, and one-class classifiers.

Furthermore, we discuss evaluation metrics, benchmark datasets, and real-world applications of anomaly detection such as fraud detection, intrusion detection, medical diagnosis, and fault detection in industrial processes. Finally, we outline promising future directions for research in anomaly detection, including the integration of multi-modal data, transfer learning, and the development of robust algorithms capable of handling high-dimensional and streaming data in dynamic environment.

TABLE OF CONTENTS

List of Figures

List of Tables

CHAPTER – 1

INTRODUCTION

1.1 PROBLEM STATEMENT

In real-time, companies stream data from their data warehouses or storage facilities. In the process of streaming, there is a very high chance that data gets corrupted. To target such corrupted data while data is being streamed is a difficult process. To tackle this problem, we have decided to work with this Anomaly Detection model. Anomalies are the discrepancies or anomalies in the data. Using this approach, we can tackle a very existing, difficult problem.

1.2 MOTIVATION

This problem is not only challenging but also, its solution is very much applicable to other fine-grained classification problems, helping companies around the world while streaming their data, to reduce their “outliers” or anomalies. Ultimately, we found this form of anomaly detection to be the most realistic and most important solution for a major problem faced by multiple companies.

1.3 SCOPE

The application addresses the challenge of anomaly detection through a combination of Denoising Autoencoders (DAE) for image anomaly detection and Background Subtraction with Centroid Tracking for video anomaly detection. The system leverages neural networks within the autoencoder architecture to encode and denoise images, calculate reconstruction errors, and classify anomalies. For video anomaly detection, the application employs background subtraction techniques to detect moving objects and uses centroid tracking to identify unusual motion patterns.

The proposed system is structured into three main phases: Pre-processing, Training, and Classification.

Pre-processing Phase:

- **Image Data:** The dataset undergoes normalization using statistical techniques such as Mean, Median, and Mode to standardize the input data. This ensures that the neural network receives consistent and reliable data for training.
- **Video Data:** Background subtraction is applied to separate moving objects from the static background. Morphological operations are used to clean the foreground mask, removing noise and enhancing the detection of significant objects.

Training Phase:

- **Denoising Autoencoder (DAE) for Images:**
  - The DAE model is trained on normalized image data. The encoder compresses the input data into a lower-dimensional representation, learning the essential features of the normal data while reducing noise.
  - The decoder reconstructs the original image from the encoded representation. The model is trained to minimize the reconstruction error between the input and the output.
- **Background Subtraction and Centroid Tracking for Videos:**
  - After background subtraction, the system identifies contours in the foreground mask to detect moving objects.
  - Centroid tracking is employed to monitor the movement of these objects across frames, calculating the direction and speed of motion.

Classification Phase:

- **Image Anomalies:**
  - Reconstruction errors are calculated by comparing the original and reconstructed images. High reconstruction errors indicate poor reconstruction, suggesting the presence of anomalies.
  - Density-Based Spatial Clustering of Applications with Noise (DBSCAN) is used to cluster normal and anomalous data points based on the reconstruction errors.

- **Video Anomalies:**
    - The direction and speed of tracked objects are analyzed. Anomalous frames are identified by detecting objects moving in unexpected directions or speeds.

    - Identified anomalous frames are displayed and saved for further analysis.

1.4 OUTLINE

The Neural Network architecture has three phases. The phases being:

- Preprocessing
- Training
- Classification

The dataset is split into training and testing datasets. The neural network learns from the training dataset and then the model is evaluated on the testing dataset.

CHAPTER – 2

LITERATURE SURVEY

EXISTING SYSTEM:

Existing systems for real-time anomaly detection in streaming data environments provide efficient solutions by combining traditional statistical methods, machine learning algorithms, and specialized techniques. Leveraging stream processing platforms like Apache Kafka or Apache Flink, these systems offer timely detection of anomalies and anomalies across diverse domains. However, they face drawbacks including reliance on historical data for model training, susceptibility to concept drift, scalability challenges due to computational complexity, and difficulty in interpreting and validating detected anomalies in real-time. Despite these limitations, ongoing research aims to enhance the effectiveness of these systems for real-time anomaly detection in streaming data environments.

LIMITATIONS OF EXISTING SYSTEM:

- **Data Requirements:** Extensive labeled training data is necessary, which may not always be available, affecting the model's ability to generalize to unseen data.

- **Computational Complexity:** Deep learning models used in the system are computationally intensive, posing scalability challenges, especially in environments with limited resources.

- **Dynamic Environments:** Video anomaly detection relying on background subtraction and centroid tracking can struggle in dynamic or cluttered settings, leading to false positives or missed anomalies.

- **Subtle Anomalies:** The system may not effectively capture subtle behavioral anomalies, which can be crucial in certain applications.

- **Maintenance and Updates:** Continuous updates and fine-tuning are required to maintain the system's efficacy, making it resource-intensive in terms of maintenance.

- **Preprocessing Dependency:** The performance of the system is heavily dependent on the quality of preprocessing and feature extraction stages, which can vary significantly based on the input data and environment.

PROPOSED SYSTEM:

The proposed system for anomaly detection integrates Neural Networks, Denoising Autoencoders (DAE) for Image Anomaly Detection, and OpenCV for Video Anomaly Detection to overcome previous limitations. By utilizing Denoising Autoencoders, the system effectively learns and reconstructs normal image patterns, identifying anomalies through significant reconstruction errors. For video anomaly detection, background subtraction combined with centroid tracking efficiently detects and tracks objects, identifying anomalies based on irregular motion patterns. The system leverages advanced clustering techniques to classify anomalies accurately and ensures robust real-time processing. This approach addresses issues like noise, concept drift, and scalability, promising enhanced performance and reliability for real-time anomaly detection across various domains.

ADVANTAGES OF PROPOSED SYSTEM:

- **Enhanced Anomaly Detection:** Utilizes Neural Networks and Denoising Autoencoders (DAE) for effective anomaly detection in images by identifying significant reconstruction errors.

- **Efficient Video Analysis:** Incorporates OpenCV for real-time video anomaly detection using background subtraction and centroid tracking, enabling precise detection of irregular motion patterns.

- **Noise Resilience:** Denoising Autoencoders help in filtering out noise from the data, leading to more accurate anomaly detection in images.

- **Concept Drift Handling:** The system can adapt to changes in data patterns over time, addressing issues related to concept drift and maintaining accuracy.

- **Scalability:** Designed to handle large volumes of data efficiently, ensuring robust performance even with increasing data loads.

- **Real-time Processing:** Capable of processing data in real-time, making it suitable for applications that require immediate anomaly detection.

- **Advanced Clustering:** Leverages sophisticated clustering techniques to accurately classify and distinguish anomalies from normal data.

- **Versatility:** Applicable across various domains, providing reliable anomaly detection for diverse types of image and video data.

- **Reduced False Positives:** Combines multiple detection methods to minimize false positives and enhance the reliability of the detected anomalies.

CHAPTER – 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 Overall Description:

- **Image Anomaly Detection:** Deep Autoencoders (DAE) are employed to learn a compressed representation of input images, minimizing the reconstruction error between the original and reconstructed images. Anomalies are identified based on deviations from learned patterns, as indicated by unusually high reconstruction errors.

- **Video Anomaly Detection**: Background subtraction techniques are utilized to separate moving foreground objects from a relatively static background in video frames. Anomalies are detected by identifying sudden changes or unexpected movements in the foreground objects, signalling abnormal events or behaviour.

3.2. Operating Environment:

HARDWARE REQUIREMENTS

- **Processor** : Intel Pentium® Dual Core Processor (Min)
- **Speed** : 2.9 GHz (Min)
- **RAM** : 2 GB (Min)
- **Hard Disk** : 2 GB (Min)

SOFTWARE REQUIREMENTS

- **Operating System** : Windows 7 (Min)
- **Front End** : HTML, CSS, JS (Min)
- **Back End** : Python
- **Database** : Spreadsheets (Min)

User Characteristics:

The primary users of this Anomaly Detection System are data scientists, machine learning engineers, and analysts working across various domains that require anomaly detection in images and videos. These users range from academic researchers exploring new detection methods to industry professionals implementing real-time monitoring solutions. Users are expected to have varying levels of expertise in machine learning and image processing, necessitating a system that offers both powerful detection capabilities and an intuitive, user-friendly interface. Additionally, users value accuracy, efficiency, and the ability to handle large datasets, emphasizing the importance of robust pre-processing, training, and anomaly detection functionalities. The system is designed to cater to these diverse user needs by providing clear documentation, streamlined workflows, and scalable, high-performance processing to support detailed analysis and decision-making processes.

Functional Requirements:

The system must be capable of detecting anomalies in both images and videos using advanced machine learning techniques. For image anomaly detection, the system leverages Denoising Autoencoders (DAE) to preprocess and analyze images, identifying outliers based on reconstruction error. For video anomaly detection, the system employs background subtraction and centroid tracking to identify unusual movements or behaviors in the video frames. The system should offer the ability to save and display detected anomalies, providing users with visual insights into the detected anomalies. It must be able to handle high-volume data streams efficiently, ensuring real-time processing and analysis. Additionally, the system should include user feedback mechanisms to continuously improve its detection accuracy and user experience. The interface should be user-friendly, providing clear instructions and visual representations of the results.

Admin Functionality:

- Admin plays a major role in this project.
- Admin has the authority to manipulate the data in the database.
- Adding new training data to improve the system's accuracy and reliability.

- Monitoring system performance and making necessary adjustments to the detection algorithms.
- Verifying the detected anomalies and ensuring the system's output is accurate and reliable.
- Updating the system with new algorithms or improvements to enhance its detection capabilities.
- Providing support and guidance to users, ensuring they can effectively utilize the system for their anomaly detection needs.

3.4 Non-Functional Requirements:

3.4.1 Performance Requirements:

Performance requirements refer to static numerical requirements placed on the interaction between the users and the software.

***Response Time*:**

Average response time shall be less than 5 sec.

Recovery Time:

In case of system failure, the redundant system shall resume operations within 30 secs. Average repair time shall be less than 45 minutes.

Start-Up/Shutdown Time:

The system shall be operational within 1 minute of starting up.

Utilization of Resources:

The system shall store in the database no more than 200 different colleges with room for improvement.

3.4.2 Safety Requirements:

-NA-

3.4.3 Security Requirements:

The model will be running on a secure website i.e., an HTTPS website and also on a secure browser such as Google Chrome, Brave, etc.

3.4.4 Software Quality Attributes: *Reliability:*

The system shall be reliable i.e., in case the webpage crashes, progress will be saved.

Availability:

The website will be available to all its users round the clock i.e., they can access the website at any time.

Security:

The model will be running on a secure website i.e., an HTTPS website, and on a secure browser such as Google Chrome, Brave, etc.

Maintainability:

The model shall be designed in such a way that it will be very easy to maintain in the future. Our model is a web-based system and will depend much on the web server. However, the web application will be designed using proper database modeling along with extensive documentation which will make it easy to develop, troubleshoot, and maintain in the future.

Usability:

The interfaces of the system will be user friendly enough that every user will be able to use it easily.

Scalability:

The system will be designed in such a way that it will be extendable. If more species or algorithms are going to be added in the system, then it would easily be done.

The same system can also be developed to become a mobile application rather than just a website.

CHAPTER – 4

SYSTEM DESIGN

Use case Diagram:

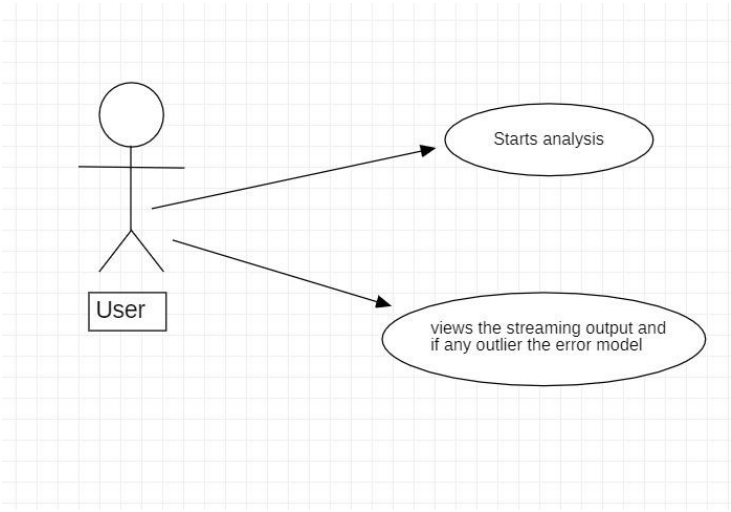


fig 4.1: Use case diagram for User

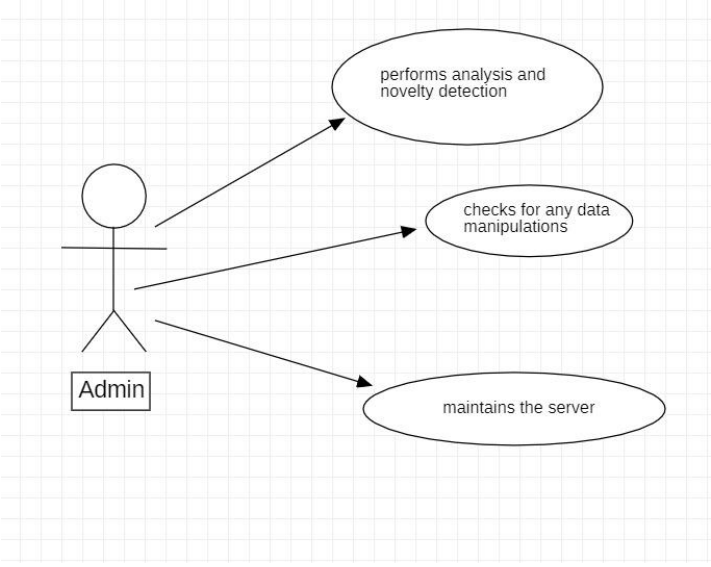


fig 4.2: Use case diagram for Admin

Class Diagram:

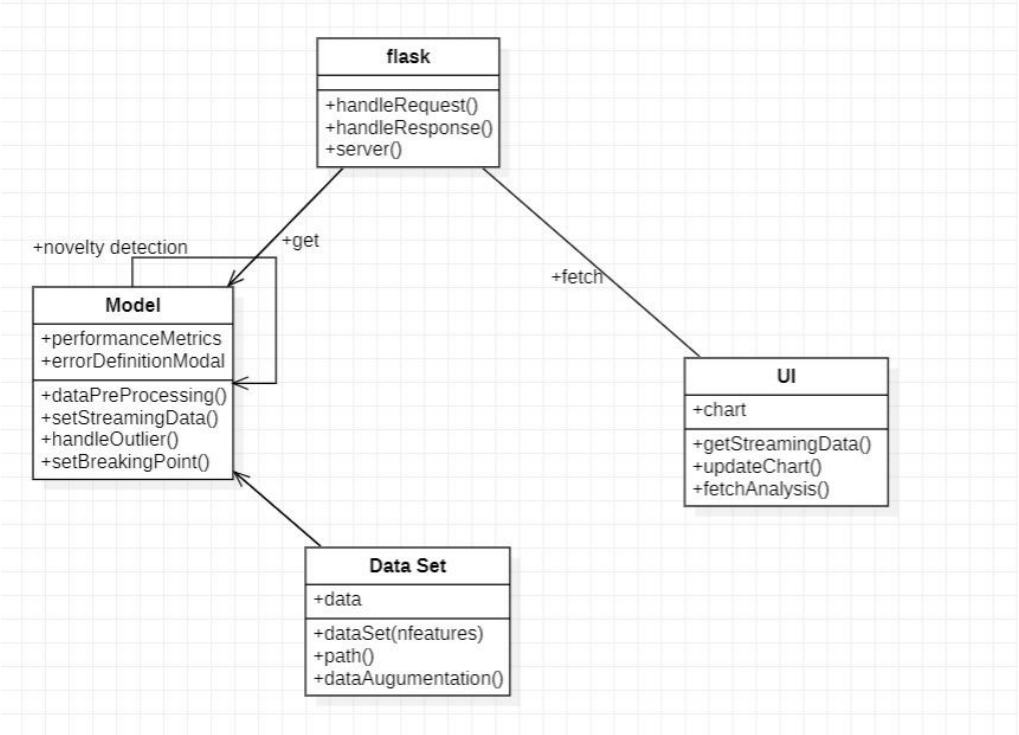


Fig.4.3: Class diagram

Sequential Diagram:

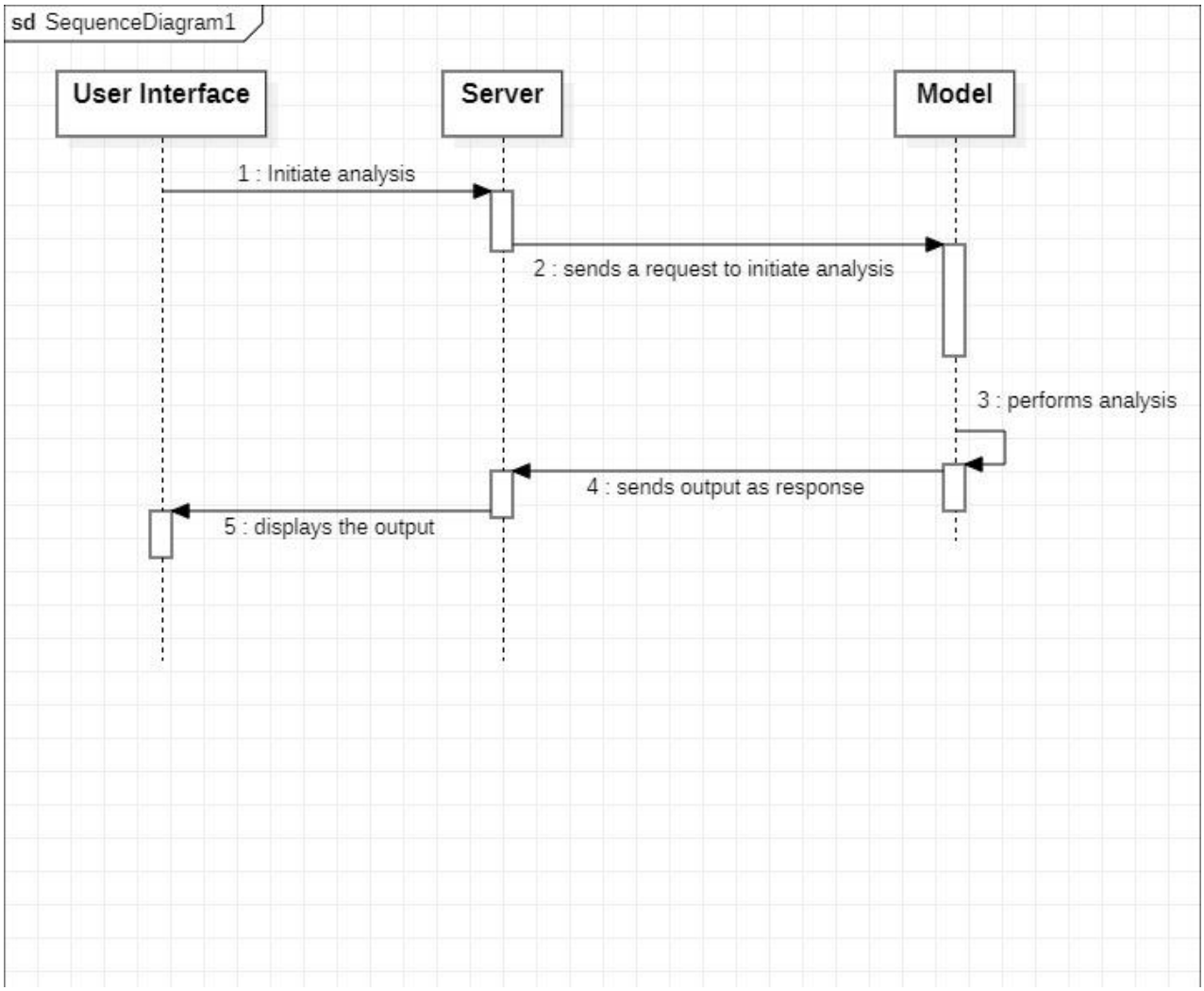


Fig4.4 : Sequence Diagam

Architecture Diagram:

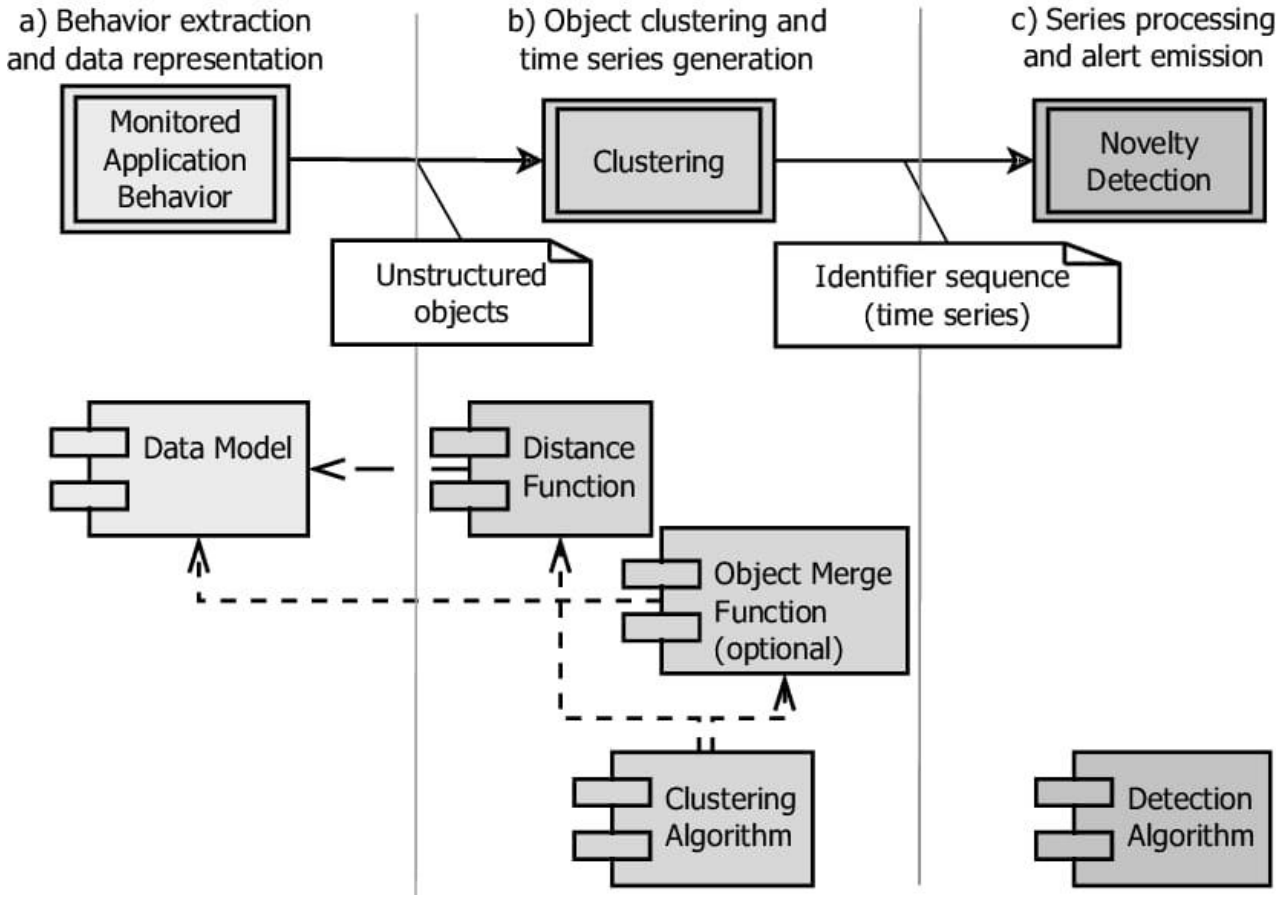


Fig4.5 : Architecture Diagam

CHAPTER – 5

IMPLEMENTATION

5.1 COMPONENTS

1. Image Anomaly Detection:

- **Data Preparation and Preprocessing**: The MNIST dataset serves as the foundation, meticulously preprocessed to normalize pixel values and reshape images.

- **Model Architecture Definition:** A Convolutional Autoencoder (CAE) model architecture is meticulously crafted to facilitate image reconstruction.

- **Model Training and Fine-Tuning:** The CAE model undergoes rigorous training, fine-tuned to extract subtle features crucial for anomaly detection.

- **Error Calculation and Thresholding:** Reconstruction errors are meticulously calculated, enabling the establishment of a threshold for anomaly determination.

- **Anomaly Identification:** Anomalies are precisely pinpointed based on reconstruction errors surpassing the established threshold, ensuring robust anomaly detection capabilities.

- **Visualization and Storage:** Anomalies are visually represented and systematically stored for subsequent analysis, enhancing interpretability and facilitating further investigations.

2. Video Anomaly Detection:

- **Input Handling and Selection**: An intuitive graphical interface facilitates the selection of video files, streamlining the input process.

- **Frame-by-Frame Analysis:** Each frame of the selected video undergoes meticulous scrutiny, ensuring thorough anomaly detection.

- **Foreground Extraction:** Advanced background subtraction techniques meticulously extract foreground objects, enabling precise anomaly detection.

- **Motion Analysis and Anomaly Detection:** Motion between consecutive frames is meticulously analyzed, enabling the accurate identification of anomalous behavior.

- **Visualization and Storage:** Anomalies are visually presented and systematically stored, facilitating comprehensive review and analysis to inform decision-making processes.

5.2 CODE

Image_Anomaly_Detection.ipynb

```
import tensorflow as tf

from tensorflow import keras

from tensorflow.keras.datasets import mnist

from sklearn.cluster import DBSCAN

import matplotlib.pyplot as plt

import numpy as np

import cv2


# Load MNIST data

(train_images, _), (test_images, _) = mnist.load_data()


# Pre-processing (normalize pixel values between 0 and 1)

train_images = train_images.astype('float32') / 255.0

test_images = test_images.astype('float32') / 255.0


# Reshape to include channels (assuming grayscale images)

train_images = train_images.reshape(-1, 28, 28, 1)

test_images = test_images.reshape(-1, 28, 28, 1)

# Define DAE model (replace with your specific architecture)

def create_dae_model():

    inputs = keras.Input(shape=(28, 28, 1))

    encoded = keras.layers.Conv2D(16, (3, 3), activation='relu', padding='same')(inputs)

    encoded = keras.layers.MaxPooling2D((2, 2), padding='same')(encoded)

    decoded = keras.layers.Conv2DTranspose(16, (3, 3), activation='relu',
padding='same')(encoded)
```

```
    decoded = keras.layers.UpSampling2D((2, 2))(decoded)
    decoded = keras.layers.Conv2D(1, (3, 3), activation='sigmoid', padding='same')(decoded)
    return keras.Model(inputs=inputs, outputs=decoded)

# Create and compile DAE model
model = create_dae_model()
model.compile(loss='binary_crossentropy', optimizer='adam')

# Train the DAE model (adjust epochs and batch size)
n_epochs = 30
batch_size = 64
model.fit(train_images, train_images, epochs=n_epochs, batch_size=batch_size)

# Save the trained model
model.save('mnist_dae_model.h5')
model = keras.models.load_model('mnist_dae_model.h5')


# Get reconstructions for test data
test_reconstructions = model.predict(test_images)

# Calculate reconstruction error (mean squared error)
reconstruction_error = np.mean(np.square(test_images - test_reconstructions), axis=(1, 2, 3))

# Define anomaly threshold based on your data analysis (e.g., 95th percentile)
threshold = np.quantile(reconstruction_error, 0.95)
```

```
# Identify anomalies based on reconstruction error exceeding threshold
anomaly_indices = np.where(reconstruction_error > threshold)[0]

# Get three random anomaly indices
anomaly_indices_to_display = np.random.choice(anomaly_indices, size=3, replace=False)

# Display and save anomaly images
for i, idx in enumerate(anomaly_indices_to_display):
    plt.figure()
    plt.imshow(test_images[idx].reshape(28, 28), cmap='gray')
    plt.title(f'Anomaly Image {i+1}')
    plt.axis('off')
    plt.savefig(f'anomaly_image_{i+1}.jpg', bbox_inches='tight')

plt.show()
```

Video_Anomaly_Detection.ipynb

```
import cv2
import numpy as np
import random
import tkinter as tk
from tkinter import filedialog

def select_video_file():
    root = tk.Tk()
    root.withdraw()
    file_path = filedialog.askopenfilename(title="Select Video File", filetypes=[("Video files", "*.mp4;*.avi")])
```

```
    return file_path

def find_anomalous_frame(video_path):
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        print("Error: Could not open video file")
        return

    ret, prev_frame = cap.read()
    if not ret:
        print("Error: Could not read the first frame")
        return

    # Initialize previous centroid
    prev_centroid = None

    # Create background subtractor
    bg_subtractor = cv2.createBackgroundSubtractorMOG2()

    anomalous_frames = []

    while True:
        ret, frame = cap.read()
        if not ret:
            break

        # Convert frame to grayscale
```

```
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

        # Apply background subtraction
        fgmask = bg_subtractor.apply(gray)

        # Apply morphological operations to remove noise
        fgmask = cv2.morphologyEx(fgmask, cv2.MORPH_OPEN, np.ones((5,5),np.uint8))
        fgmask = cv2.morphologyEx(fgmask, cv2.MORPH_CLOSE, np.ones((20,20),np.uint8))

        # Find contours
        contours, _ = cv2.findContours(fgmask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

        # Draw bounding boxes and calculate centroids
        for contour in contours:
            x, y, w, h = cv2.boundingRect(contour)
            centroid = (x + w // 2, y + h // 2)
            cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 0), 2)
            cv2.circle(frame, centroid, 5, (0, 0, 255), -1)

            # Check direction of motion
            if prev_centroid:
                dx = centroid[0] - prev_centroid[0]
                dy = centroid[1] - prev_centroid[1]
                if dx * dy < 0:  # Opposite direction motion
                    anomalous_frames.append(frame)

            # Update previous centroid
```

```
            prev_centroid = centroid

        # Display frame
        cv2.imshow("Anomaly Frame", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):  # Press 'q' to stop
            break

    cap.release()
    cv2.destroyAllWindows()

    if anomalous_frames:
        random_frame = random.choice(anomalous_frames)
        cv2.imshow("Random Anomalous Frame", random_frame)
        cv2.imwrite("anomaly_frame.jpg", random_frame)
        print("Random anomalous frame saved as 'anomaly_frame.jpg'")
        cv2.waitKey(0)
    else:
        print("No anomalous frames found")

if __name__ == "__main__":
    video_path = select_video_file()
    if video_path:
        find_anomalous_frame(video_path)
    else:
        print("No video file selected")
```

CHAPTER – 6

TESTING

6.1 TEST CASES

Test Case to check whether the required Software is installed on the systems

Test Case ID:	1
Test Case Name:	Required Software Testing
Purpose:	To check whether the required Software is installed on the systems
Input:	Enter python command
Expected Result:	Should Display the version number for the python
Actual Result:	Displays python version
Failure	If the python environment is not installed, then the Deployment fails

Table 6.1.1 python Installation verification

Test Case to check Program Integration Testing

Test Case ID:	2
Test Case Name:	Programs Integration Testing
Purpose:	To ensure that all the modules work together
Input:	All the modules should be accessed.
Expected Result:	All the modules should be functioning properly.
Actual Result:	All the modules should be functioning properly.
Failure	If any module fails to function properly, the implementation fails.

Table 6.1.2 python Programs Integration Testing

Case to Collect Dataset and Load the Dataset

Test Case ID:	3
Test Case Name:	Running of streamlit application
Purpose:	To query the chatbot
Input:	Query related to criminal law or civil law
Expected Result:	Response related to the query is displayed
Actual Result:	Response related to the query is displayed
Failure	If the response is not retrieved, application is not working

Table 6.1.3 Collect Dataset and Load the Dataset

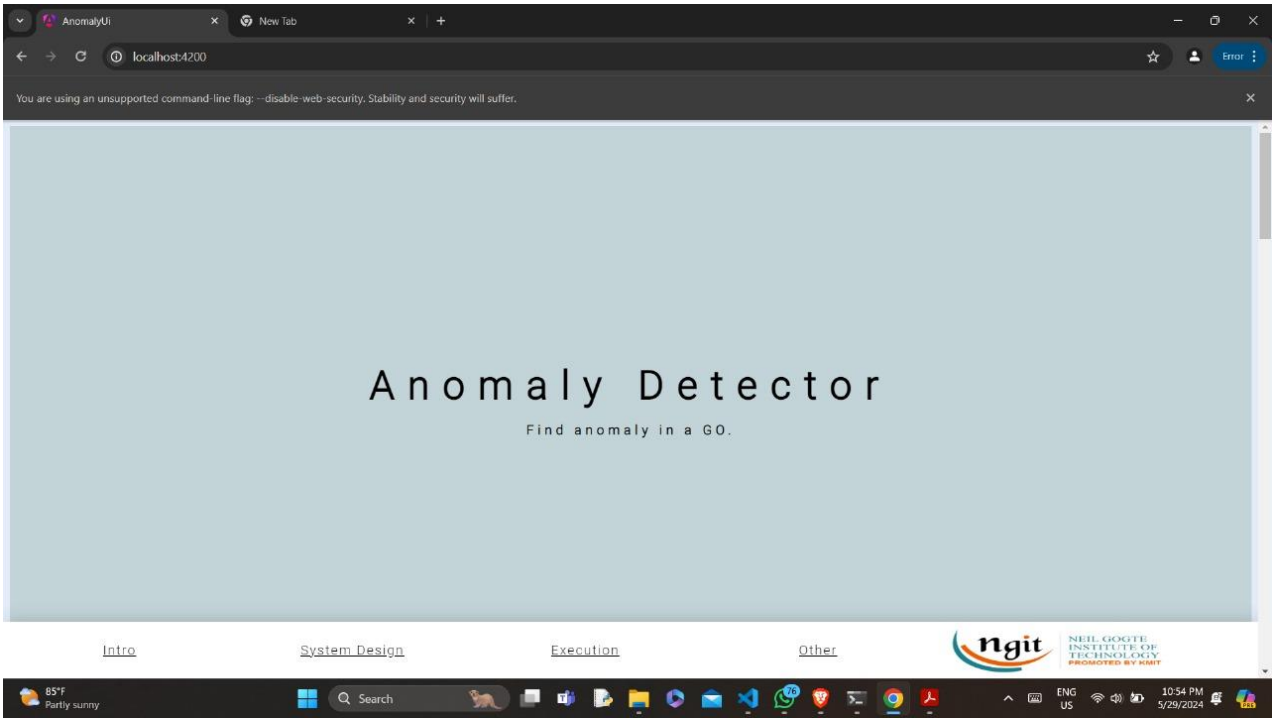
Test Case to check whether the response is appropriate

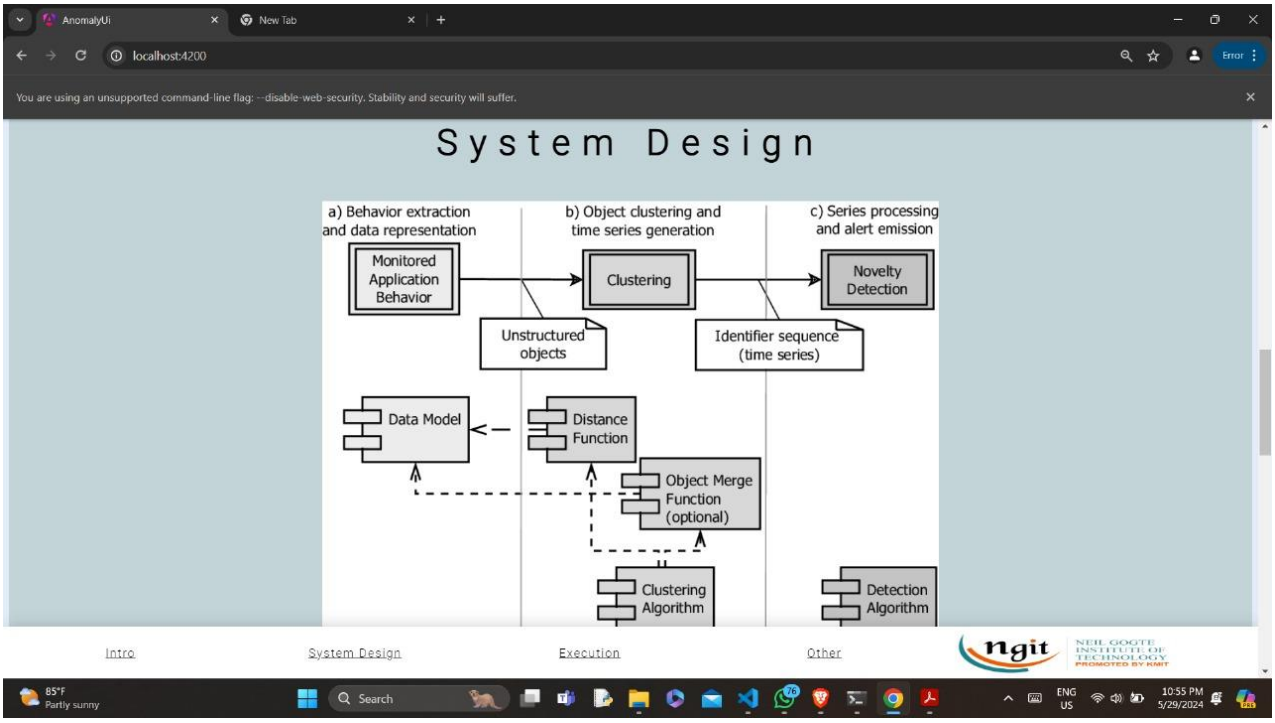
Test Case ID:	4
Test Case Name:	Response to the question is related to civil law or criminal law
Purpose:	Verifying the response from the chatbot
Input:	Query related to either civil law or criminal law
Expected Result:	Response related to the query is displayed
Actual Result:	Response related to the query is displayed
Failure	Response is not related to the query provided

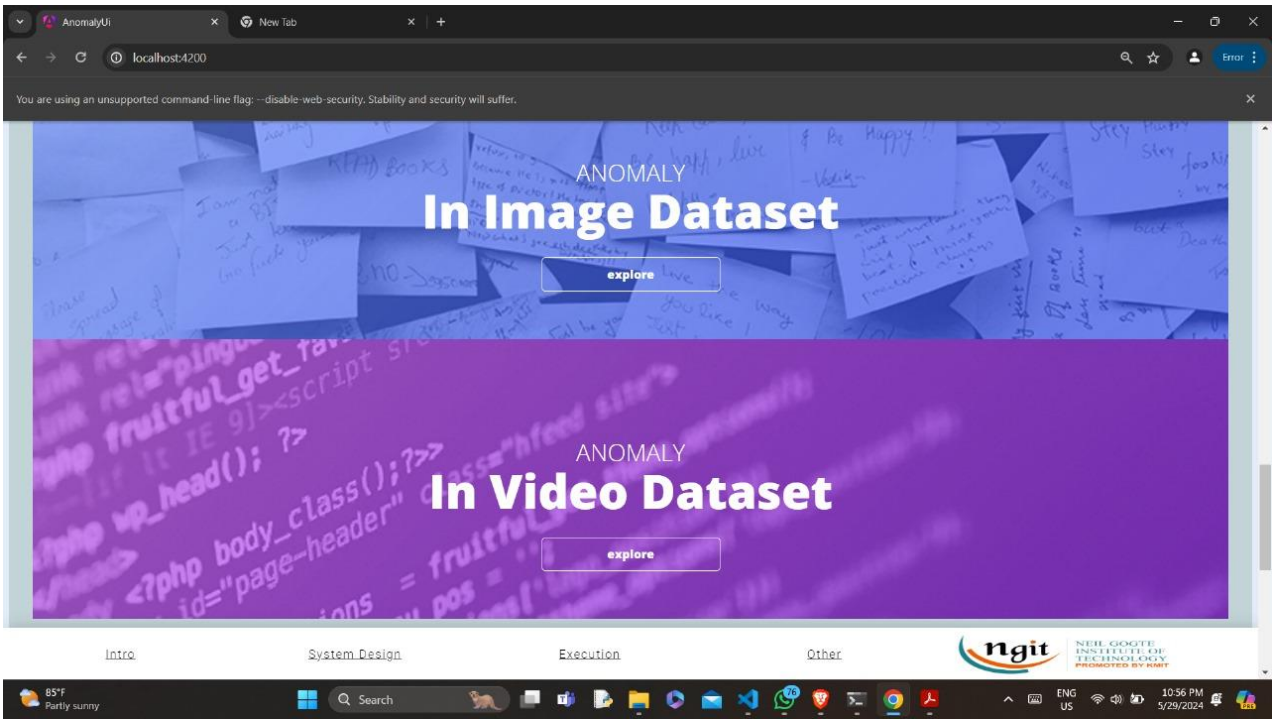
Table 6.1.4 Species Recognition

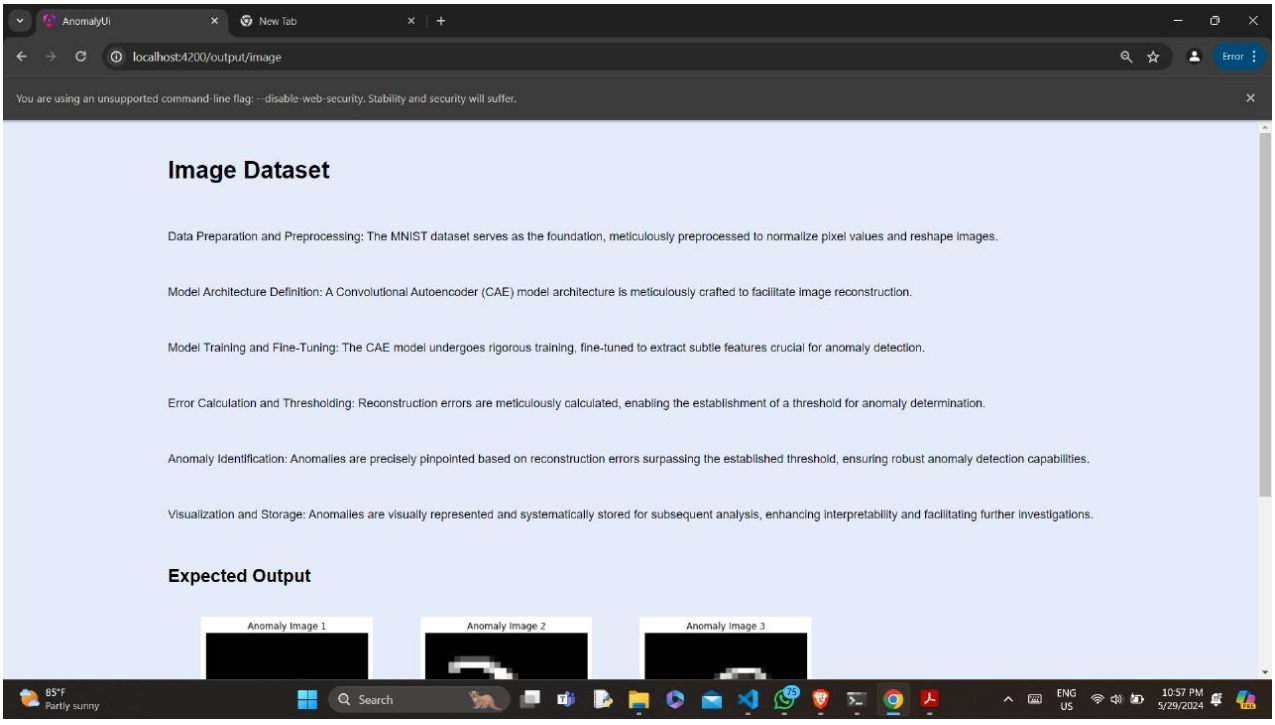
CHAPTER – 7

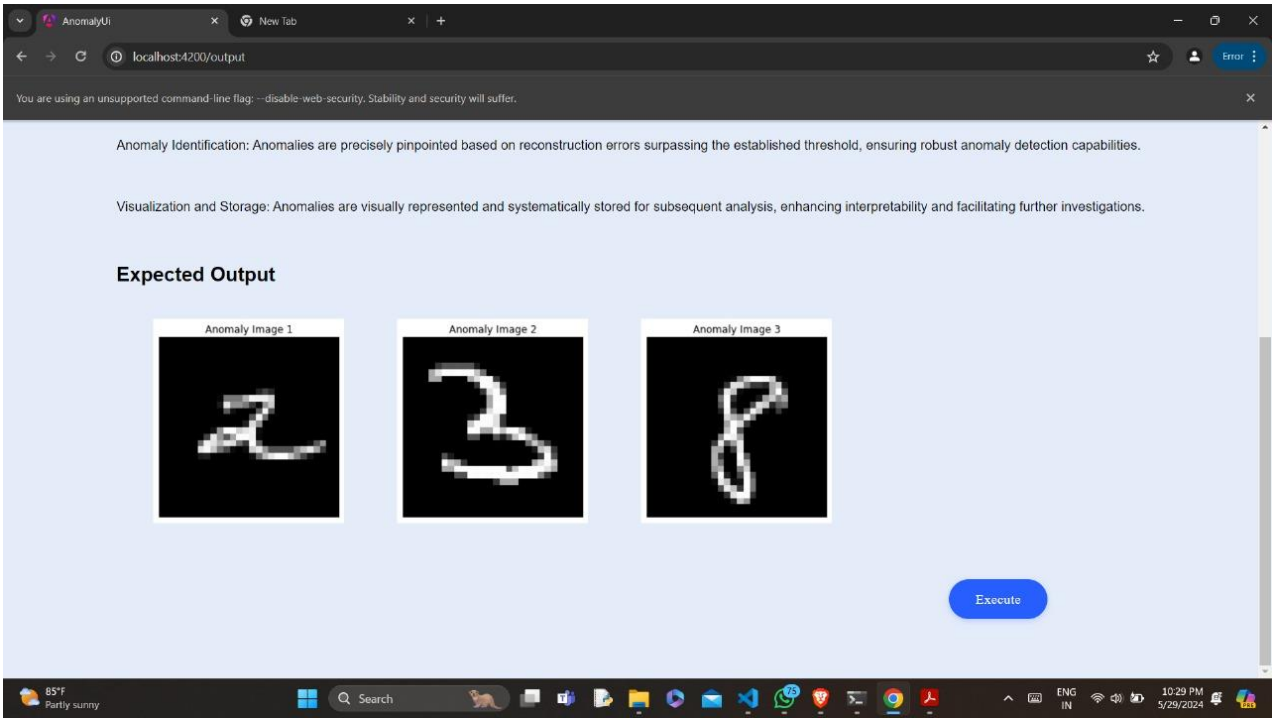
SCREENSHOTS

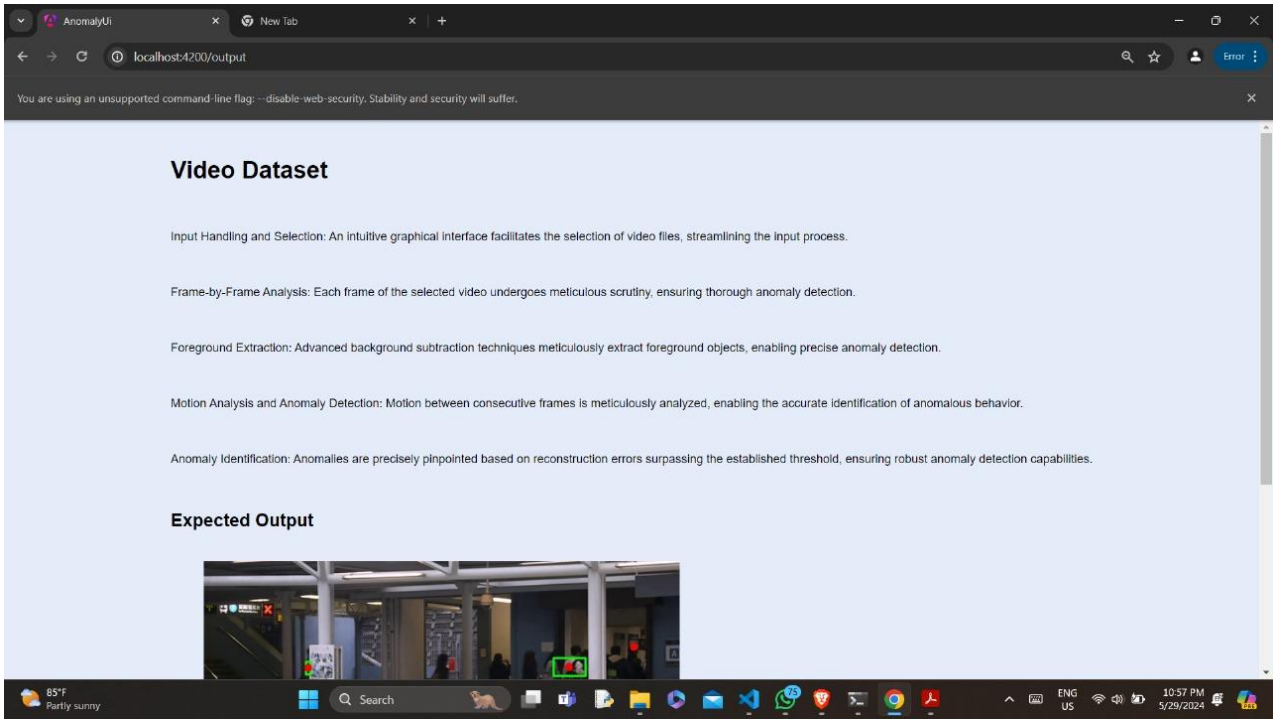


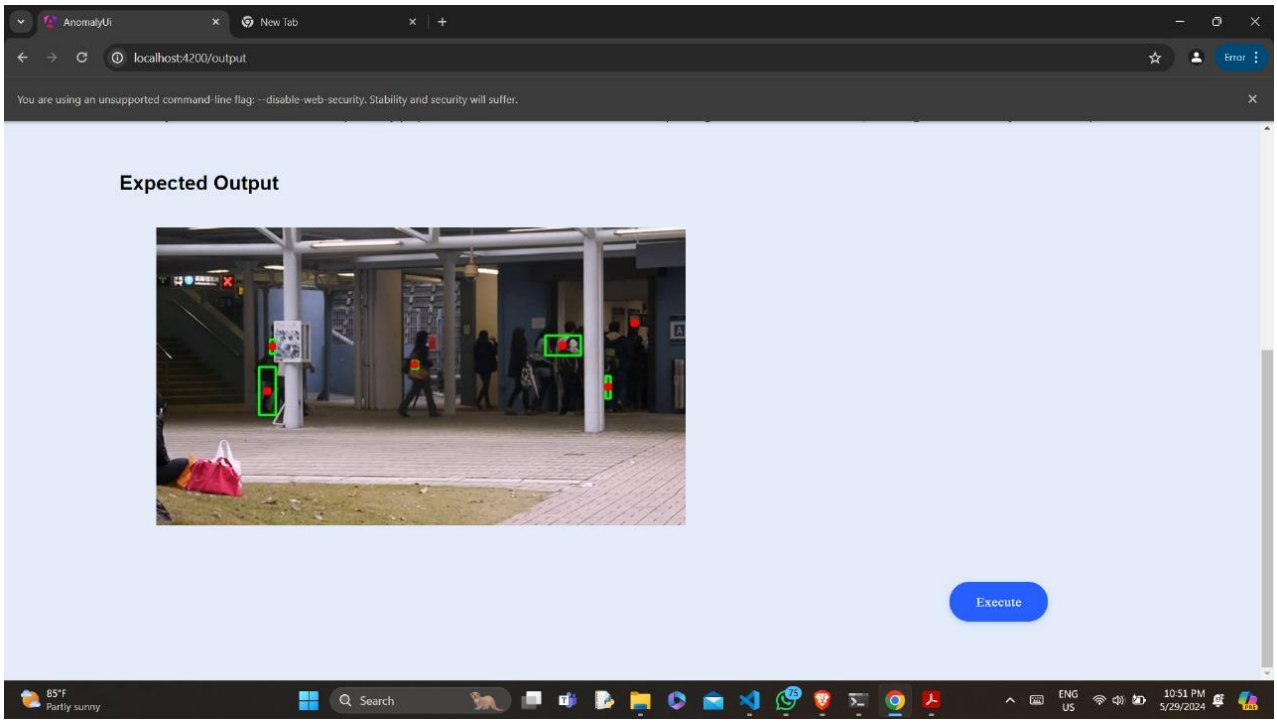



AnomalyUi
New Tab
localhost:4200
You are using an unsupported command-line flag: --disable-web-security. Stability and security will suffer.
ANOMALY
In Image Dataset
explore
ANOMALY
In Video Dataset
explore
Intro
System Design
Execution
Other
ngit
NEIL GOGTE INSTITUTE OF TECHNOLOGY
PROMOTED BY KMIT
85°F
Partly sunny
Search
ENG
US
10:56 PM
5/29/2024


AnomalyUi
New Tab
localhost:4200/output/image
You are using an unsupported command-line flag: --disable-web-security. Stability and security will suffer.
Image Dataset
Data Preparation and Preprocessing: The MNIST dataset serves as the foundation, meticulously preprocessed to normalize pixel values and reshape images.
Model Architecture Definition: A Convolutional Autoencoder (CAE) model architecture is meticulously crafted to facilitate image reconstruction.
Model Training and Fine-Tuning: The CAE model undergoes rigorous training, fine-tuned to extract subtle features crucial for anomaly detection.
Error Calculation and Thresholding: Reconstruction errors are meticulously calculated, enabling the establishment of a threshold for anomaly determination.
Anomaly Identification: Anomalies are precisely pinpointed based on reconstruction errors surpassing the established threshold, ensuring robust anomaly detection capabilities.
Visualization and Storage: Anomalies are visually represented and systematically stored for subsequent analysis, enhancing interpretability and facilitating further investigations.
Expected Output
Anomaly Image 1
Anomaly Image 2
Anomaly Image 3
85°F
Partly sunny
Search
ENG
US
10:57 PM
5/29/2024


AnomalyUi
New Tab
localhost:4200/output
Error
You are using an unsupported command-line flag: --disable-web-security. Stability and security will suffer.
Anomaly Identification: Anomalies are precisely pinpointed based on reconstruction errors surpassing the established threshold, ensuring robust anomaly detection capabilities.
Visualization and Storage: Anomalies are visually represented and systematically stored for subsequent analysis, enhancing interpretability and facilitating further investigations.
Expected Output
Anomaly Image 1
Anomaly Image 2
Anomaly Image 3
Execute
85°F
Partly sunny
Search
ENG
IN
10:29 PM
5/29/2024


AnomalyUi
New Tab
localhost:4200/output
Error
You are using an unsupported command-line flag: --disable-web-security. Stability and security will suffer.
Video Dataset
Input Handling and Selection: An intuitive graphical interface facilitates the selection of video files, streamlining the input process.
Frame-by-Frame Analysis: Each frame of the selected video undergoes meticulous scrutiny, ensuring thorough anomaly detection.
Foreground Extraction: Advanced background subtraction techniques meticulously extract foreground objects, enabling precise anomaly detection.
Motion Analysis and Anomaly Detection: Motion between consecutive frames is meticulously analyzed, enabling the accurate identification of anomalous behavior.
Anomaly Identification: Anomalies are precisely pinpointed based on reconstruction errors surpassing the established threshold, ensuring robust anomaly detection capabilities.
Expected Output
85°F
Partly sunny
Search
ENG
US
10:57 PM
5/29/2024


AnomalyUi
New Tab
localhost:4200/output
Error
You are using an unsupported command-line flag: --disable-web-security. Stability and security will suffer.
Expected Output
Execute
85°F
Partly sunny
Search
ENG
US
10:51 PM
5/29/2024

CHAPTER – 8

CONCLUSION AND FUTURE SCOPE

This project demonstrates two approaches for anomaly detection: utilizing a reconstruction-based method for static images and a background subtraction-based method for videos. The image anomaly detection successfully identifies unusual digits within the MNIST dataset using a trained autoencoder. The video anomaly detection leverages background subtraction and motion analysis to detect sudden changes in a video, potentially indicating anomalies. Both methods offer valuable tools for identifying deviations from expected patterns.

There's significant room for improvement and expansion upon these initial approaches. For image anomaly detection, exploring different autoencoder architectures and anomaly scoring methods could enhance the system's ability to identify diverse anomalies. Additionally, incorporating techniques like image segmentation or domain-specific training data could further refine anomaly detection for more complex image types. In video anomaly detection, including features like object detection or optical flow analysis could enable the system to distinguish between normal object movement and actual anomalies. Moreover, exploring advanced anomaly scoring methods that combine multiple factors could lead to more robust and reliable anomaly detection in real-world video scenarios.

BIBLIOGRAPHY

Code References:

- OpenCV Documentation on Background Subtraction: https://docs.opencv.org/3.4/db/d5c/tutorial_py_bg_subtraction.html
- OpenCV Documentation on cv2.findContours: https://docs.opencv.org/3.4/df/d0d/tutorial_find_contours.html
- OpenCV Documentation on cv2.morphologyEx: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html

Advanced Video Anomaly Detection Techniques:

- **Background Subtraction Enhancements:**
  - Bouwmans, T., et al. "Background modeling using mixture of Gaussians." In Proc. IEEE Int'l Conf. on Image Processing (ICIP), vol. 1, pp. II-746-II-749 (2001). A paper exploring background subtraction with MOG2
- **Object Detection and Tracking:**
  - Explore object detection models like YOLOv5 or SSD for integration.
  - Consider libraries like OpenCV's `cv2.calcOpticalFlowFarneback` or deep learning approaches for optical flow analysis.
- **Anomaly Scoring and Classification:**
  - Scikit-learn: https://scikit-learn.org/ offers tools for building anomaly scoring systems (e.g., Support Vector Machines, Random Forests).
  - TensorFlow/PyTorch: https://www.tensorflow.org/, https://pytorch.org/ enable development of deep learning models for anomaly classification.

Additional Resources:

- OpenCV Tutorials: https://docs.opencv.org/4.x/d9/df8/tutorial_root.html provide in-depth guides on advanced background subtraction and optical flow analysis.

APPENDIX A: TOOLS AND TECHNOLOGIES

This appendix provides an overview of the various tools and technologies used in the development of our intelligent chatbot system. Each tool and technology played a crucial role in ensuring the functionality, efficiency, and user experience of the chatbot.

- **OpenCV (Open-Source Computer Vision Library):**
    - **Core Functionalities:** OpenCV forms the backbone of the current system, providing functionalities for background subtraction (anomaly detection), contour detection (object isolation), morphological operations (noise reduction), and centroid tracking (motion analysis).
- Python**:**
    - **Programming Language:** Python serves as the foundation for code development, enabling the integration of OpenCV's functionalities and potential future libraries.
- **NumPy:**
    - **Efficient Array Manipulation:** While not used in the current implementation, NumPy offers efficient array manipulation capabilities. It can significantly accelerate computations during video processing tasks like background subtraction and feature extraction (for future enhancements).
- **TensorFlow/Keras:**
    - **Machine Learning Integration:** TensorFlow and Keras are powerful libraries for building and training machine learning models. Future enhancements could involve integrating these libraries to develop a deep learning-based anomaly detection system. This could involve training a model on labeled video datasets to learn normal behavior patterns and identify deviations as anomalies.

These tools and technologies collectively contributed to the successful development and deployment of our intelligent chatbot system, providing a robust, secure, and user-friendly solution for handling anomaly detection in images and videos.